NAME: V Vishwa Teja ID NO: 2520090033

SKILL WEEK 10

1. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_received = 0;

void signal_handler(int sig, siginfo_t *info, void *context)
{
    signal_received = sig;

    printf("\nSignal received: %d\n", sig);
    printf("Sender PID: %d\n", info->si_pid);
    printf("Process PID: %d\n", getpid());
}

int main()
{
    struct sigaction action;

    memset(&action, 0, sizeof(action));

    action.sa_sigaction = signal_handler;
    action.sa_flags = SA_SIGINFO;

    sigemptyset(&action.sa_mask);

    sigaction(SIGINT, &action, NULL);
    sigaction(SIGTERM, &action, NULL);
    sigaction(SIGUSR1, &action, NULL);

    printf("Process PID: %d\n", getpid());
    printf("Signal handler registered.\n");
    printf("Waiting for signals...\n");

    while (1)
    {
        pause();

        if (signal_received == SIGTERM)
        {
            printf("SIGTERM received. Exiting safely.\n");
            break;
        }

        signal_received = 0;
    }
```

```
    }

    return 0;
}
```

Main terminal OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill10.1.c -o skill10.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill10.1
Process PID: 2010
Signal handler registered.
Waiting for signals...

Signal received: 10
Sender PID: 2018
Process PID: 2010

Signal received: 2
Sender PID: 2018
Process PID: 2010

Signal received: 15
Sender PID: 2018
Process PID: 2010
SIGTERM received. Exiting safely.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Another terminal commands:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGUSR1 2010
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGINT 2010
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ kill -SIGTERM 2010
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

2. To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

pid_t foreground_pid = -1;

void sigint_handler(int sig)
{
    if (foreground_pid > 0)
    {
        printf("\nShell forwarding SIGINT to foreground job %d\n",
               foreground_pid);

        kill(-foreground_pid, SIGINT);
    }
    else
    {
        printf("\nShell protected from SIGINT\n");
    }
}

int main()
{
    struct sigaction action;
    pid_t pid;
    int status;

    memset(&action, 0, sizeof(action));
    action.sa_handler = sigint_handler;
    sigemptyset(&action.sa_mask);

    sigaction(SIGINT, &action, NULL);

    printf("Shell PID: %d\n", getpid());
    printf("Starting foreground job: sleep 30\n");

    pid = fork();

    if (pid == 0)
    {
        setpgid(0, 0);
        signal(SIGINT, SIG_DFL);
        execlp("sleep", "sleep", "30", NULL);
        perror("exec");
        exit(1);
    }
```

```
    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    setpgid(pid, pid);
    foreground_pid = pid;

    printf("Foreground job PID: %d\n", pid);
    printf("Press Ctrl+C to interrupt the foreground job.\n");

    waitpid(pid, &status, 0);

    foreground_pid = -1;

    if (WIFSIGNALED(status))
    {
        printf("Foreground job terminated by signal %d\n",
               WTERMSIG(status));
    }
    else if (WIFEXITED(status))
    {
        printf("Foreground job exited normally.\n");
    }

    printf("Shell is still running.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill10.2.c -o skill10.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill10.2
Shell PID: 2169
Starting foreground job: sleep 30
Foreground job PID: 2170
Press Ctrl+C to interrupt the foreground job.
^C
Shell forwarding SIGINT to foreground job 2170
Foreground job exited normally.
Shell is still running.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

3. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[50];
    char state[20];
} Job;

Job job;

pid_t foreground_pid = -1;

void sigtstp_handler(int sig)
{
    if (foreground_pid > 0)
    {
        printf("\nSuspending foreground job %d\n",
               foreground_pid);

        kill(-foreground_pid, SIGTSTP);
    }
}

void display_job()
{
    printf("\nJob ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("PGID: %d\n", job.pgid);
    printf("Command: %s\n", job.command);
    printf("State: %s\n", job.state);
}

int main()
{
    struct sigaction action;
    int status;

    memset(&action, 0, sizeof(action));
    action.sa_handler = sigtstp_handler;
    sigemptyset(&action.sa_mask);
```

```
    sigaction(SIGTSTP, &action, NULL);

    job.job_id = 1;
    strcpy(job.command, "sleep 30");
    strcpy(job.state, "Running");

    pid_t pid = fork();

    if (pid == 0)
    {
        setpgid(0, 0);
        signal(SIGTSTP, SIG_DFL);
        execlp("sleep", "sleep", "30", NULL);
        perror("exec");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    job.pid = pid;
    job.pgid = pid;

    setpgid(pid, pid);

    foreground_pid = pid;

    printf("Foreground job started.\n");
    display_job();

    printf("\nPress Ctrl+Z to suspend the job.\n");

    waitpid(pid, &status, WUNTRACED);

    foreground_pid = -1;

    if (WIFSTOPPED(status))
    {
        strcpy(job.state, "Stopped");
        printf("\nJob suspended.\n");
        display_job();
    }

    printf("\nPress Enter to resume the job.\n");
    getchar();
```

```c
    printf("Sending SIGCONT...\n");

    kill(-job.pgid, SIGCONT);

    strcpy(job.state, "Running");

    display_job();

    printf("\nWaiting for job completion...\n");

    waitpid(pid, &status, 0);

    strcpy(job.state, "Done");

    printf("\nJob completed.\n");
    display_job();

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill10.3.c -o skill10.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill10.3
Foreground job started.

Job ID: 1
PID: 2234
PGID: 2234
Command: sleep 30
State: Running

Press Ctrl+Z to suspend the job.
^Z
Suspending foreground job 2234

Press Enter to resume the job.

Sending SIGCONT...

Job ID: 1
PID: 2234
PGID: 2234
Command: sleep 30
State: Running

Waiting for job completion...

Job completed.

Job ID: 1
PID: 2234
PGID: 2234
Command: sleep 30
State: Done
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Crate a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t shell_pgid;
    pid_t child_pid;
    int status;

    shell_pgid = getpgrp();

    printf("Shell PID: %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);
    printf("Shell terminal foreground PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    child_pid = fork();

    if (child_pid == 0)
    {
        setpgid(0, 0);

        printf("\nChild PID: %d\n", getpid());
        printf("Child PGID: %d\n", getpgrp());

        execlp("sleep", "sleep", "5", NULL);

        perror("exec");
        exit(1);
    }

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    setpgid(child_pid, child_pid);

    printf("\nCreated process group.\n");
    printf("Child PID: %d\n", child_pid);
    printf("Child PGID: %d\n", child_pid);

    printf("\nTransferring terminal control to child group...\n");

    if (tcsetpgrp(STDIN_FILENO, child_pid) == -1)
```

```c
        perror("tcsetpgrp");

    printf("Terminal foreground PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    printf("Sending SIGCONT to child group...\n");

    kill(-child_pid, SIGCONT);

    waitpid(child_pid, &status, 0);

    printf("\nChild completed.\n");

    printf("Restoring terminal control to shell...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
        perror("tcsetpgrp");

    printf("Terminal foreground PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    printf("Terminal ownership restored.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill10.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill10.4.c -o skill10.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill10.4
Shell PID: 2290
Shell PGID: 2290
Shell terminal foreground PGID: 2290

Created process group.
Child PID: 2291
Child PGID: 2291

Transferring terminal control to child group...
Terminal foreground PGID: 2291
Sending SIGCONT to child group...

Child PID: 2291
Child PGID: 2291

Child completed.
Restoring terminal control to shell...

[1]+  Stopped                 ./skill10.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```